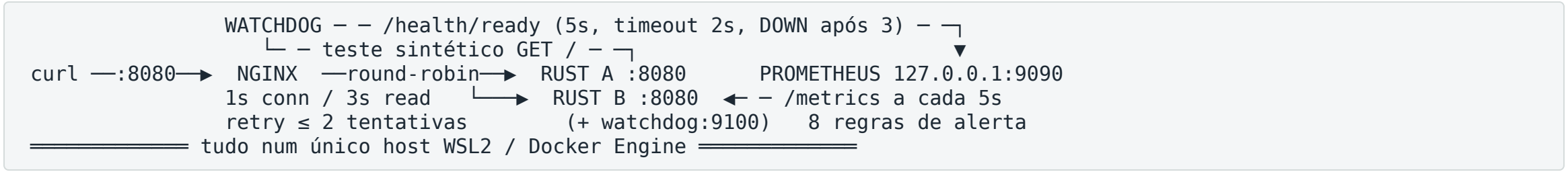
1. Arquitetura e pontos únicos de falha (A)



SPOF	Se cair…	O que temos
Nginx	serviço inteiro fora	teste sintético + alerta crítico
Watchdog	ninguém declara DOWN	alerta `MonitorParado` (Prometheus vigia)
Prometheus	nenhum alerta — e ninguém avisa	**declarado como SPOF residual**
Bug comum A+B	balanceamento não ajuda	alerta de taxa de erro
Host / Docker	tudo	limite do laboratório

Rust A/B **não** são SPOF (redundância 2×). Detalhe: `docs/A_arquitetura_e_spof.md`

2. O que já roda (B e C) — evidência de 02/11

```
$ docker compose ps
nginx       Up (healthy)  0.0.0.0:8080->80/tcp      ← única porta de usuário
prometheus  Up (healthy)  127.0.0.1:9090->9090/tcp  ← só local
rust-a      Up (healthy)  8080/tcp                  ← só rede interna
rust-b      Up (healthy)  8080/tcp
watchdog    Up (healthy)  9100/tcp
```

Teste	Resultado
20 requisições	**A 10 · B 10**, todas 200
Mesmo `request_id`	1 linha no Nginx + 1 no Rust
`rust-a` não pronto	`live` **200**, `ready` **503** · usuário 10/10 com 200 via B
Log do Nginx	`upstream_status=503, 200` → recusou em A, refeito em B
`/metrics`, `/admin` pela porta 8080	404

Evidências: `evidencias/preliminar/01…03`, 10/10 em `05` (após recuperação) e `05b` · Nginx comentado: `nginx/nginx.conf`

3. Métricas e alertas planejados (D)

SLIs: disponibilidade (teste sintético ≥ 99%/5 min) · erro de usuário ≤ 2%/2 min · p95 ≤ 1 s

Métrica	Mede
`watchdog_target_up`	disponibilidade vista **de fora**
`app_requests_total{status}`	taxa de erro e tráfego por nó
`app_request_duration_seconds`	latência p95
`app_inflight_requests`	saturação
`watchdog_last_probe_timestamp_seconds`	o monitor está vivo?

Alerta	Limiar	Por quê
NoDown (alta)	DOWN (3 falhas × 5 s) + 5 s	1 perda isolada não alarma
TaxaErro5xx (alta)	> 2% em 2 min, por 30 s	abaixo é ruído; acima o usuário sente
MonitorParado (média)	última sondagem > 30 s	6 intervalos; folga p/ saltos de relógio

- 5 complementares (sintético, p95, coleta, reinício em loop, flapping)

4. Hipóteses e experimentos (início de E)

H1 — Parar A → serviço continua; 1–3 requisições ~1 s (connect timeout) — **CONFIRMADA** `20/20 → 200 via B` · `3 req a 1,09 s` (05c) · watchdog DOWN em **14 s** · `NoDown` ativo em **22 s** (05) · alertas resolvidos após `start` · `RestartCount=0`

H2 — A lento (2,5 s) → watchdog diz DOWN (timeout 2 s) mas Nginx ainda usa A (read 3 s): **os dois vão discordar** — previsto Hora 6

H3 — Parar watchdog → usuário não percebe; `MonitorParado` em ~35 s; queda real **sem** `NoDown` nesse intervalo — previsto Hora 6

Previstos	**E2b atraso 3,5 s (retry amplifica carga) · E4 `/erro` 500 sem retry · E6 tempo de retorno × `resolver valid`**

Achado: após `start rust-a`, watchdog vê UP em **2–3 s**, mas o tráfego leva **até ~10 s** para voltar (medido: ≤ 4,6 s, 9,4 s, 10,5 s; depende da fase da re-resolução DNS do Nginx). Monitor e balanceador **discordam** por até ~8 s.

5. Riscos, dificuldades e plano até 12/11

Dificuldades já encontradas (e corrigidas, com evidência antes/depois): sonda estourava o timeout por DNS (3,6 s > 2 s) · alerta de erro com **falso positivo** (contava 503 que o usuário nunca viu) · flapping falso por `increase()` · healthcheck do Nginx dependia dos nós · **relógio do WSL2 salta −2,4 s/+5,1 s**

Riscos: SLI de erro medido no nó, não na borda · Prometheus sem vigia · build do Rust na máquina da aula · tudo num só host

Data	Entrega
06–08/11	Hora 6: E2, E2b, E3, E4, E6 com previsão → evidência
em aula	Hora 7: incidente (checklist pronto em `docs/F`)
09–10/11	Relatório F, README final, melhoria priorizada: SLI na borda (exporter Nginx)
11/11	Ensaio cronometrado da demo + vídeo plano B
12/11	Entrega A–G + demo ao vivo